

TP - Programmation Fonctionnelle

Exploration d'un graphe en TypeScript

Objectif : implementer des parcours de graphe (DFS et BFS) en TypeScript, dans un style strictement fonctionnel : fonctions pures, recursion, et donnees immuables (par discipline).

Contraintes generales (obligatoires)

- Aucune boucle (for, while, forEach)
- Aucune fonction map / filter / reduce (ni equivalents)
- Aucune mutation (pas de push/pop, pas d'affectation sur une structure partagee)
- Aucun if / else (utiliser des expressions : operateur ternaire, appels de fonctions, etc.)
- Recursivite autorisee
- Fonctions pures uniquement (pas d'I/O, pas d'etat global)

Donnees fournies

Le graphe est represente par une liste d'adjacence : pour chaque noeud, une liste d'aretes sortantes.

```
export type NodeId = string;

export type Edge = {
  to: NodeId;
  cost: number;
};

export type Graph = {
  adj: { [node: string]: Edge[] };
};

export const neighbors = (g: Graph, n: NodeId): Edge[] =>
  g.adj[n] ?? [];
```

Q1 - Outils fonctionnels sur les listes

Attendu : écrire des utilitaires pour manipuler des tableaux sans boucles et sans map/filter/reduce. Vous reutiliserez ces fonctions partout ensuite.

Signatures à implémenter

```
export const isEmpty = <T>(xs: T[]): boolean => { /* ... */ };

export const head = <T>(xs: T[]): T => { /* ... */ };

export const tail = <T>(xs: T[]): T[] => { /* ... */ };

export const prepend = <T>(x: T, xs: T[]): T[] => { /* ... */ };

export const append = <T>(xs: T[], x: T): T[] => { /* ... */ };

// concat(a, b) = a ++ b (récursif)
export const concat = <T>(a: T[], b: T[]): T[] => { /* ... */ };
```

Guide algorithmique (concat)

- Si a est vide : resultat = b
- Sinon : resultat = prepend(tete(a), concat(reste(a), b))

Q2 - Ensemble des noeuds visites (visited)

Attendu : gérer les cycles. visited indique quels noeuds ont déjà été traités. Il doit être manipulé de façon immuable (retourner une nouvelle valeur).

Signatures à implémenter

```
export type VisitSet = { [node: string]: true };

export const hasVisited = (vs: VisitSet, n: NodeId): boolean => { /* ... */ };

// markVisited retourne un NOUVEL objet (ne jamais modifier vs)
export const markVisited = (vs: VisitSet, n: NodeId): VisitSet => { /* ... */ };
```

Guide algorithmique

- hasVisited(vs, n) vérifie la présence de n (ex: vs[n] === true)
- markVisited(vs, n) : si déjà présent => vs ; sinon => { ...vs, [n]: true }

Rappel - DFS (Depth-First Search)

DFS = parcours en profondeur. On explore un chemin le plus loin possible avant de revenir en arriere. Implementation typique : une pile (LIFO) comme frontiere.

Schema DFS (pile)

- frontiere = [start]
- depiler n (tete de pile)
- si n est la cible : succes
- si n est visite : ignorer
- sinon : marquer visite et empiler ses voisins DEVANT le reste de la pile
- repeter recursivement jusqu'a trouver target ou epuiser la pile

Q3 - Atteignabilite par DFS

Attendu : reachableDFS renvoie true si target est atteignable depuis start, sinon false. Gerer les cycles avec visited.

Signature a implementer

```
export const reachableDFS = (  
  g: Graph,  
  start: NodeId,  
  target: NodeId  
): boolean => { /* ... */ };
```

Guide algorithmique

Implementer une fonction recursive go(frontiere, visited). Cas d'arret : frontiere vide => false.
Sinon : depiler un noeud n, tester n === target, ignorer si visite, sinon marquer visite et empiler les voisins devant le reste.

Q4 - Table des parents (parent) + reconstruction de chemin

Attendu : enregistrer, pour chaque noeud decouvert, son parent (le noeud depuis lequel il a ete decouvert). Cela permet de reconstruire un chemin en remontant depuis target jusqu'a start.

Signatures a implementer

```
export type ParentMap = { [node: string]: NodeId | null };

export const hasParent = (pm: ParentMap, n: NodeId): boolean => { /* ... */ };

// Fixe le parent seulement si child n'a pas encore de parent
export const setParent = (
  pm: ParentMap,
  child: NodeId,
  parent: NodeId | null
): ParentMap => { /* ... */ };

// Reconstruit [start, ..., target] ou null
export const buildPathFromParents = (
  pm: ParentMap,
  start: NodeId,
  target: NodeId
): NodeId[] | null => { /* ... */ };
```

Guide algorithmique (reconstruction)

- Si target n'a pas de parent enregistre : null
- Sinon : remonter parent[target], puis parent[parent[target]], etc. jusqu'a start
- Construire la liste dans le bon ordre (souvent : construire a l'envers puis inverser)

Q5 - Trouver un chemin par DFS (un chemin valide)

Attendu : trouver un chemin valide (pas forcément le plus court) de start a target. Faire un DFS, et fixer parent[child] au moment ou child est decouvert.

Signature a implementer

```
export const findPathDFS = (
  g: Graph,
  start: NodeId,
  target: NodeId
): NodeId[] | null => { /* ... */ };
```

Guide algorithmique

- DFS avec une pile frontiere et un visited
- Initialiser parents[start] = null
- A la decouverte d'un voisin child (non visite, non deja decouvert), enregistrer setParent(parents, child, current)
- Arrêter quand target est decouvert (ou quand la frontiere est vide)
- Reconstruire avec buildPathFromParents

Rappel - BFS (Breadth-First Search)

BFS = parcours en largeur. On explore le graphe par couches : tous les noeuds a distance 1 (en nombre d'aretes) avant ceux a distance 2, etc.

Pourquoi BFS donne le plus court chemin (en nombre d'aretes)

BFS utilise une file (FIFO) : on defile en tete et on enfile en fin. Ainsi, les noeuds sont explores par distance croissante depuis start. Le premier parent enregistre pour un noeud correspond donc a un chemin minimal (en nombre d'aretes).

Schema BFS (file)

- file = [start]
- defiler n (tete de file)
- ajouter en fin les voisins non encore decouverts
- repeter recursivement jusqu'a target ou file vide

Q6 - Plus court chemin (BFS)

Attendu : retourner un chemin de longueur minimale (en nombre d'aretes) entre start et target, ou null.

Signature a implementer

```
export const shortestPathBFS = (  
  g: Graph,  
  start: NodeId,  
  target: NodeId  
) : NodeId[] | null => { /* ... */ };
```

Guide algorithmique

- BFS avec une file frontiere et un visited
- Initialiser parents[start] = null
- A la decouverte d'un voisin child, enregistrer son parent UNE SEULE FOIS (premiere decouverte)
- Enfiler les voisins en fin de file (FIFO) pour assurer la minimalite
- A la fin : buildPathFromParents(parents, start, target)

Donnees de test

```
export const sample: Graph = {
  adj: {
    A: [{ to: "B", cost: 1 }, { to: "C", cost: 1 }],
    B: [{ to: "D", cost: 1 }],
    C: [{ to: "D", cost: 1 }, { to: "E", cost: 1 }],
    D: [{ to: "B", cost: 1 }], // cycle
    E: [],
  },
};

// Attendus :
// reachableDFS(sample, "A", "E") === true
// reachableDFS(sample, "E", "A") === false
// findPathDFS(sample, "A", "D") -> chemin valide
// shortestPathBFS(sample, "A", "D") -> chemin minimal (3 noeuds)
```